
Probeklausur 5

Grundlagen der Programmierung

Studiengänge:	PEB & MVB	Autor:	Luis Staudt
Semester:	Sommersemester 2026	Schwierigkeit:	schwer
Bearbeitungszeit:	90 Minuten	Gesamtpunkte:	90

*Bearbeiten Sie alle fünf Aufgaben. Achten Sie auf sauberen, kompilierfähigen Java-Quelltext.
Hilfsmittel: OpenBook, keine elektronischen Geräte.*

Aufgabe 1: Struktogramm in Java umsetzen**(24 Punkte)**

Das folgende Struktogramm liest Zahlen ein, klassifiziert sie über ihre Teilersumme (vollkommen / abundant) und sucht anschließend die *erste* Primzahl im Feld. Beide Teile verwenden eine **geschachtelte** Schleife.

Setzen Sie das Struktogramm **vollständig in ein lauffähiges Java-Programm** um.

lies n ein; lege Feld <code>werte</code> mit n Plätzen an	
für $i = 0, 1, \dots, n - 1$	
	lies <code>werte[i]</code> ein
für $i = 0, 1, \dots, n - 1$	
<code>tsum = 0;</code>	
für $t = 1, \dots, \text{werte}[i]/2$	
	<code>werte[i] % t == 0</code>
ja	nein
<code>tsum = tsum + t;</code>	<i>(nichts)</i>
<code>tsum == werte[i]</code>	
ja	nein
<code>drucke werte[i]: vollkommen</code>	<i>(nichts)</i>
<code>tsum > werte[i]</code>	
ja	nein
<code>anzAbund = anzAbund + 1;</code>	<i>(nichts)</i>
<code>i = 0; primGefunden = false;</code>	
solange (nicht <code>primGefunden</code>) und ($i < n$)	
<code>istPrim = (werte[i] >= 2);</code>	
für $t = 2, \dots, \text{werte}[i] - 1$	
	<code>werte[i] % t == 0</code>
ja	nein
<code>istPrim = false;</code>	<i>(nichts)</i>
<code>istPrim == true</code>	
ja	nein
<code>primGefunden = true;</code> <code>primPos = i;</code>	<i>(nichts)</i>
<code>i = i + 1;</code>	
<code>primGefunden == true</code>	
ja	nein
<code>drucke primPos</code>	<code>drucke: keine Primzahl</code>

Aufgabe 2: Schiefer Wurf, Größe auswählen**(14 Punkte)**

Ein Körper wird mit der Anfangsgeschwindigkeit v_0 unter dem Winkel α (in Grad) geworfen ($g = 9,81$).

Gegebene Formeln:

$$(1) \text{ Horizontalweg: } x = v_0 \cos \alpha \cdot t \quad (2) \text{ Höhe: } y = v_0 \sin \alpha \cdot t - \frac{1}{2} g t^2$$

$$(3) \text{ Steigzeit: } t_s = \frac{v_0 \sin \alpha}{g} \quad (4) \text{ Wurfweite: } w = \frac{v_0^2 \sin(2\alpha)}{g}$$

Schreiben Sie ein Programm, das die gesuchte Größe per Zahl (1 bis 4) wählen lässt, die benötigten Werte abfragt und mit der **richtigen** Formel rechnet. Behandeln Sie die Sonderfälle: bei $\alpha = 90^\circ$ ist der Horizontalweg $x = 0$ (und die Wurfweite $w = 0$), bei $\alpha = 0^\circ$ ist die Steigzeit $t_s = 0$.

Winkel im Bogenmaß. Die Java-Funktionen `Math.sin` und `Math.cos` erwarten das Bogenmaß. Rechnen Sie den Winkel mit `Math.toRadians(alpha)` bzw. $\alpha \cdot \pi / 180$ um.

Aufgabe 3: Klasse Dreieck**(20 Punkte)**

Gegeben ist die Klasse `Punkt` (Attribute `xKoordinate`, `yKoordinate` als `double`). Implementieren Sie eine Klasse `Dreieck` aus drei `Punkt`-Objekten. Das Gerüst ist vorgegeben. Ergänzen Sie:

- einen **Konstruktor** mit drei `Punkt`-Parametern.
- `berechneFlaeche()`: $A = \frac{1}{2} \left| \Delta x_{12} \cdot \Delta y_{13} - \Delta x_{13} \cdot \Delta y_{12} \right|$, wobei $\Delta x_{12} = x_2 - x_1$ usw.
- `berechneUmfang()`: Summe der drei Seitenlängen (*verwenden Sie `berechneAbstandZuPunkt`*).
- `istRechtwinklig()`: gibt `true` zurück, wenn das Dreieck (über den Satz des Pythagoras) rechtwinklig ist. Wegen Rundungsfehlern ist eine **Epsilon-Umgebung** zu verwenden.

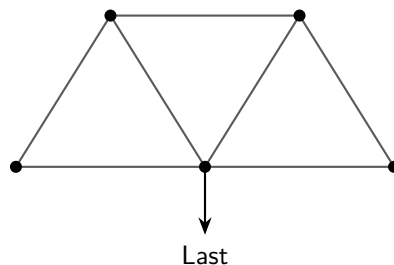
```
1 public class Dreieck {  
2     public Punkt p1, p2, p3;    // Daten  
3     // Konstruktor und Methoden ergaenzen  
4 }
```

```
1 Punkt a = new Punkt();  a.xKoordinate = 0;  a.yKoordinate = 0;  
2 Punkt b = new Punkt();  b.xKoordinate = 4;  b.yKoordinate = 0;  
3 Punkt c = new Punkt();  c.xKoordinate = 0;  c.yKoordinate = 3;  
4  
5 Dreieck d = new Dreieck(a, b, c);  
6 System.out.println("Flaeche: " + d.berechneFlaeche()); // 6.0  
7 System.out.println("Umfang:  " + d.berechneUmfang());  // 12.0  
8 if (d.istRechtwinklig())  
9     System.out.println("Das Dreieck ist rechtwinklig");
```

Aufgabe 4: Fachwerk, Knotenverschiebungen**(16 Punkte)**

Gegeben ist die aus Vorlesung und Übung bekannte Datei `fachwerk.jar`. Nach der Berechnung sollen für ein gewähltes Fachwerk zwei Auswertungen der Knotenverschiebungen programmiert werden:

- Geben Sie **alle Knoten** aus, die sich **rein vertikal** verschieben (Verschiebung in x-Richtung praktisch null, in y-Richtung von null verschieden). Existiert kein solcher Knoten, ist eine entsprechende Meldung auszugeben.
- Geben Sie zusätzlich den Knoten mit der **größten Gesamtverschiebung** samt deren Wert aus, wobei $v = \sqrt{v_x^2 + v_y^2}$.



Schematische Darstellung eines Fachwerks.

Epsilon-Umgebung beachten. Wegen Gleitkomma-Rundung ist „null“ über eine Epsilon-Umgebung zu prüfen:

$$|v_x| < \varepsilon \quad \text{und} \quad |v_y| \geq \varepsilon.$$

Benötigte Methoden: `gibAnzahlKnoten()`, `gibKnoten(int i)`, `gibVerschiebungX()`, `gibVerschiebungY()`.

Ergänzen Sie den folgenden Quelltext zu einem vollständigen, lauffähigen Programm:

```

1 public class Fachwerkaufgabe {
2     public static void main(String[] args) {
3         System.out.println("Welches Fachwerk wollen Sie rechnen (0-4)?");
4         int wahl = Tastatureingabe leseInt();
5         Fachwerk fw = new Fachwerk(wahl);
6         fw.berechneGroessen();
7         double epsilon = 1e-6;
8         // hier erweitern: rein vertikal verschobene Knoten ausgeben sowie
9         // den Knoten mit der groessten Gesamtverschiebung bestimmen
10    }
11 }
```

Mögliche Ausgabe:

- 1 Knoten 2 verschiebt sich rein vertikal
- 2 Knoten 4 verschiebt sich rein vertikal
- 3 Groesste Gesamtverschiebung: Knoten 3 mit 0.0123

Aufgabe 5: Wurfbahn in eine Datei schreiben**(16 Punkte)**

Ein Körper wird unter dem Winkel α mit v_0 geworfen ($g = 9,81$). Für die Bahnkurve gilt

$$x(t) = v_0 \cos \alpha \cdot t \quad y(t) = v_0 \sin \alpha \cdot t - \frac{1}{2} g t^2.$$

Schreiben Sie ein Programm, das ab $t = 0$ s in Schritten von 0,1 s die Werte t , $x(t)$ und $y(t)$ in die Datei `bahn.txt` schreibt. **Brechen Sie ab, sobald** $y(t) < 0$ wird (Aufprall). Hängen Sie am Ende eine Zeile mit der erreichten **Wurfweite** und der **Flugzeit** an.

Hinweise. Winkel mit `Math.toRadians(alpha)` umrechnen. Datei: `new FileWriter("bahn.txt")`, je Zeile `write(...)` mit `"\n"`, am Ende `close()`. Nutzen Sie `"\t"` für die Spaltenabstände.

Erwarteter Dateiinhalt (Auszug, $v_0 = 20$, $\alpha = 45^\circ$):

```
1 t [s]    x [m]    y [m]
2 0.0      0.0      0.0
3 0.1      1.414    1.365
4 ...
5 2.8      39.6     0.179
6 Aufprall bei t = 2.9 s, Wurfweite = 41.01 m
```